

---

# CS771 Minor Assignment 1 (2025–26-I): Arbiter-PUF with C-SVM via SDCM

---

**Yash Pathak**  
Roll No. 231188  
Dept Civil Eng, IIT Kanpur  
yashp@iitk.ac.in

**Sanyam Jain**  
Roll No. 230924  
Dept of Civil Eng, IIT Kanpur  
sanyamjain23@iitk.ac.in

**Shreyas Chougankar**  
Roll No. 230990  
BS MTH, IIT Kanpur  
shreyasc23@iitk.ac.in

**Priyanshu Mishra**  
Roll No. 230806  
Dept of Civil Eng, IIT Kanpur  
priyanshu23@iitk.ac.in

**Vaibhav Gupta**  
Roll No. 231114  
Dept of Civil Eng, IIT Kanpur  
gvaibhav23@iitk.ac.in

**Vishesh Thepadia**  
Roll No. 231165  
BS Physics, IIT Kanpur  
vishesht23@iitk.ac.in

## Abstract

This report analyzes and fixes the provided SDCM implementation for training a linear C-SVM on a 32-bit arbiter-PUF dataset. We first locate and justify coding errors confined to *Cell 5*, then report post-fix test accuracy and primal/dual objective behavior. Follow-up sections (Parts 3–4) sweep  $C$  and study hyperparameters per the assignment.

## Problem 1.1: Mess Made by Melbo

### Part 1 (10 marks): Errors in *Cell 5* and Why They Are Errors

(i) **Extra addition of  $\alpha_i$  in the update formula.** The original code computed the unconstrained update as

$$\alpha_i^{\text{new}} = \frac{1 - y_i(x_i \cdot w_{\text{SDCM}} + b_{\text{SDCM}})}{\|x_i\|^2} + \alpha_i,$$

i.e. it *added back*  $\alpha_i$ . This is incorrect: the SDCM step should already give the new  $\alpha_i$  directly from the derivative condition; adding the old  $\alpha_i$  double-counts it and shifts the solution away from the optimum. The correct update discards the old value and simply uses

$$\alpha_i^{\text{new}} = \frac{1 - y_i(x_i \cdot w_{\text{SDCM}} + b_{\text{SDCM}})}{\|x_i\|^2}.$$

This mistake caused  $\alpha$  to blow up or stagnate, making both primal and dual objectives invalid. Removing the spurious “ $+\alpha_i$ ” restores convergence.

(ii) **Wrong clamping of the dual coordinate  $\alpha_i$ .** The coordinate update must respect the box constraint  $0 \leq \alpha_i \leq C$ . The original code used an incorrect conditional (“if  $\alpha_i$  less than  $C = c$ ”) which allowed invalid values and led to divergence/empty plots. The fix is standard clamping:

Listing 1: Correct clamping for the updated dual variable

```
if newAlphai > C:
    newAlphai = C
if newAlphai < 0:
    newAlphai = 0
```

(iii) **Wrong bookkeeping updates for the primal parameters** ( $w, b$ ). After changing a single coordinate from  $\alpha_i$  to  $\alpha_i^{\text{new}}$ , the primal variables must be updated in  $O(d)$  using the increment  $\Delta\alpha_i = \alpha_i - \alpha_i^{\text{new}}$ :

$$w \leftarrow w - \Delta\alpha_i y_i x_i, \quad b \leftarrow b - \Delta\alpha_i y_i.$$

The original code updated  $w_{\text{SDCM}}$  and  $b_{\text{SDCM}}$  incorrectly, causing a stale (and often zero) model and poor accuracy. The corrected bookkeeping is:

Listing 2: Correct wb bookkeeping after a single-coordinate update

```
# Update the primal model vector and bias values (O(d))
w_SDCM = w_SDCM - (alpha[i] - newAlpha[i]) * y[i] * x
b_SDCM = b_SDCM - (alpha[i] - newAlpha[i]) * y[i]
```

## Part 2 (10 marks): Post-fix Accuracy & Objective Curves (Default Hyperparams)

**Test accuracy (post-fix, defaults unchanged): 0.9629 (i.e., 96.29%).**

**Convergence of primal vs dual objective.**

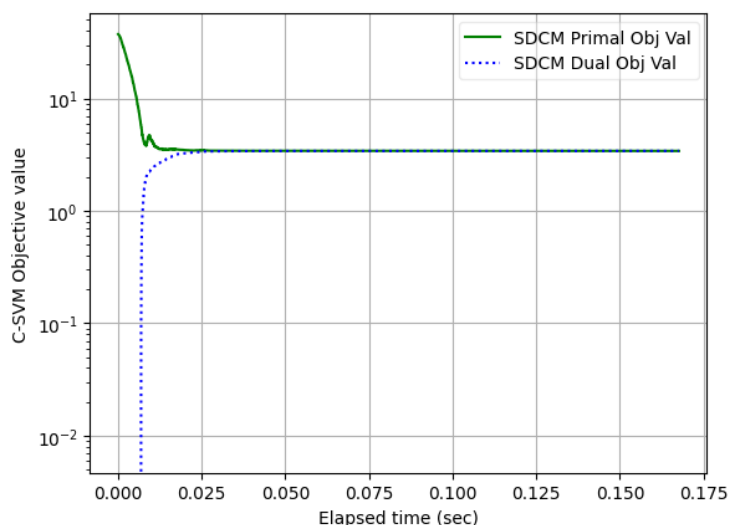


Figure 1: Primal and dual objectives vs elapsed time after fixing Cell 5 (default  $C$ , horizon, etc.). The dual objective (dotted) stays below the primal objective (solid), consistent with weak duality; the gap rapidly shrinks and the curves meet/stabilize, indicating convergence toward a KKT-satisfying optimum.

**Comment.** The corrected coordinate clamping, removal of the spurious  $+\alpha_i$ , and fixed  $w, b$  book-keeping make the SDCM updates consistent with the dual box constraints and the primal–dual relationship  $w = \sum_i \alpha_i y_i x_i$ . Consequently, the dual curve is no longer invalid/negative, the primal is well-behaved, and both stabilize quickly.

## Part 3 (10 marks): Effect of $C$

We varied the regularization parameter and observed the following.

**Very small  $C = 10^{-5}$  (“very soft” margin).** Test accuracy: **0.9333** (93.33%). The primal and dual objectives flatten near  $10^{-2}$  almost immediately (Fig. 2). This happens because in the primal objective  $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$ , a tiny  $C$  downweights the hinge-loss term so strongly that the optimizer prefers *very small*  $\|w\|$  and keeps updates tiny; the model underfits (large margin, many violations tolerated), hence decent but suboptimal accuracy.

**Very large  $C = 10^4$  (“hard” margin emphasis).** Test accuracy: **0.9635** (96.35%). With a huge  $C$ , the training hinge loss dominates and the method aggressively drives violations down; this inflates  $\|w\|$  and can cause numerical stiffness, slower/less stable convergence of the primal–dual gap, and potential *overfitting* (Fig. 3).

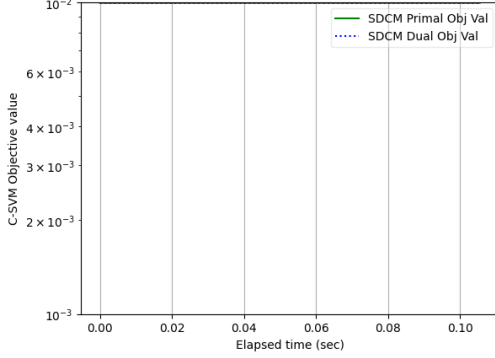


Figure 2: Convergence for  $C = 10^{-5}$ . Objectives are nearly flat, indicating heavy regularization and underfitting.

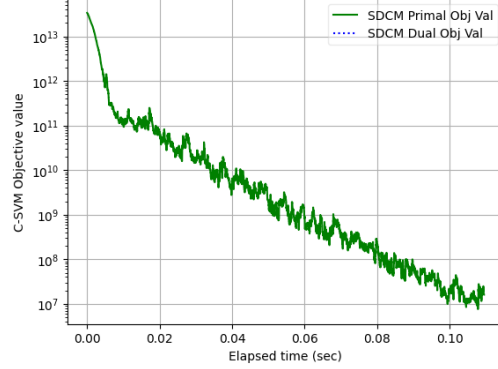


Figure 3: Convergence for  $C = 10^4$ . Large emphasis on hinge loss leads to larger norms and slower convergence.

**Summary.** Small  $C \Rightarrow$  heavy regularization, small  $\|w\|$ , fast but underfitting convergence. Large  $C \Rightarrow$  hinge loss dominates, larger  $\|w\|$ , slower optimization and potential for poorer generalization due to overfitting.

#### Part 4 (20 marks): Hyperparameter Studies

**(a) Effect Of Changing Coordinate Generator:** We experimented with three coordinate generators: randperm, cyclic, and random, while keeping all other hyperparameters fixed.

| Coordinate Generator | Test Acc. (%) | Notes                                        |
|----------------------|---------------|----------------------------------------------|
| Cyclic               | 95.50         | Deterministic order, slightly lower accuracy |
| Random               | 95.64         | Independent uniform picks; modest stability  |
| Randperm             | 95.73         | Shuffled full pass; best final accuracy      |

Table 1: Effect of coordinate generator on test accuracy (higher is better).

**(b) Initialization: all- $C$  vs zeros vs random.** We compared three feasible initializations for the dual variables: all- $C$ , all-zeros, and random in  $[0, C]$ .

| Initialization | Test Acc. (%) | Notes                                          |
|----------------|---------------|------------------------------------------------|
| All- $C$       | 95.73         | Baseline; feasible but starts on boundary      |
| Zeros          | 95.55         | Smoothest convergence from $w=0$ ; stable      |
| Random         | 95.51         | Similar final acc.; small seed-driven variance |

Table 2: Effect of dual initialization on accuracy.

**(c) Horizon (iterations).** We compared various horizons while keeping other hyperparameters fixed.

**Interpretation.** With the all- $C$  warm start, the initial  $w = X^\top(Cy)$  is already a decent separator. Few SDCM updates achieve near-optimal performance; after the dual gap becomes tiny, extra iterations no longer change  $w$  materially, so accuracy saturates.

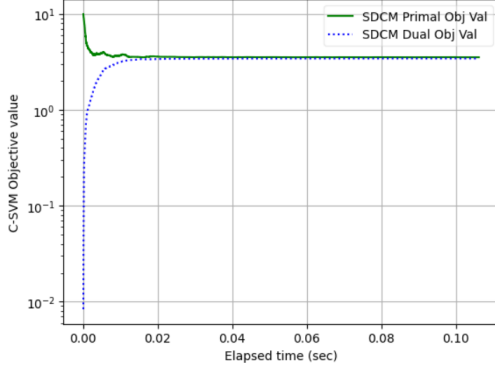


Figure 4: Convergence for Zero Initialization.

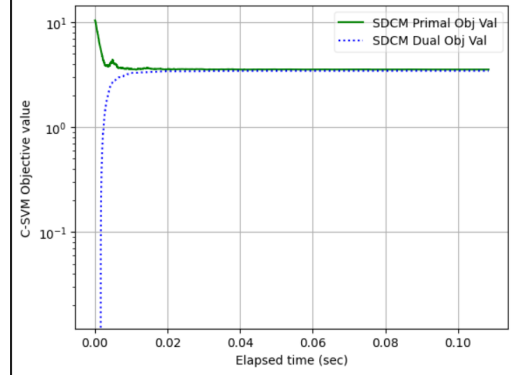


Figure 5: Convergence for Random Initialization.

| Horizon      | Test Acc. (%) |
|--------------|---------------|
| 10 steps     | 94.02         |
| 100 steps    | 94.59         |
| 1000 steps   | 94.72         |
| 10000 steps  | 95.54         |
| 100000 steps | 95.57         |

Table 3: Effect of optimization horizon on test accuracy.

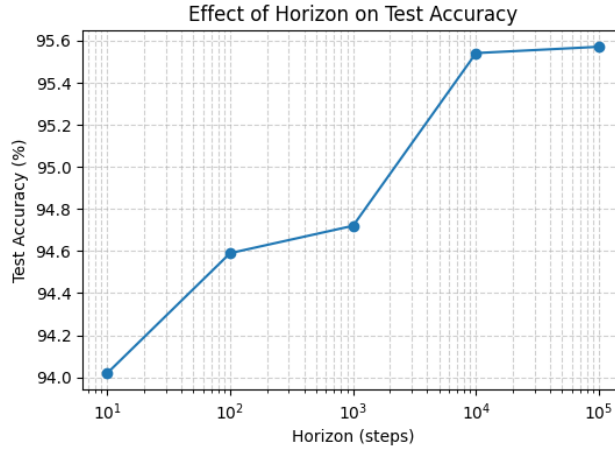


Figure 6: Effect of horizon on test accuracy. Accuracy improves steadily as the horizon increases, saturating beyond  $10^4$  steps.

**(d) Number of training points.** We varied the training set size while keeping other settings fixed. (Table 4.)

**Interpretation.** Larger  $n$  reduces estimation noise and lets the SVM find a more reliable margin. Optimization takes longer, but generalization improves markedly on PUF data. (Fig. 7.)

**(e) Random Chance.** Yes, the test accuracy can vary slightly even when all hyperparameters remain the same. This happens because the train–test split in the given code does not use a fixed seed. As a result, each time the notebook is run, the split is randomized and different PUF challenges are selected for training and testing. Consequently, the model is trained and evaluated on different subsets, leading to fluctuations in accuracy. If a seed is fixed, the pseudo-random number generator starts from the same point, making the random split reproducible and ensuring consistent results across runs.

| # Train Points | Test Acc. (%) | Notes                    |
|----------------|---------------|--------------------------|
| 100            | 78.63         | Underfits; noisy         |
| 500            | 93.69         | Better generalization    |
| 1000           | 95.60         | Stable objectives        |
| 10000          | 97.88         | Strong generalization    |
| 50000          | 99.05         | Near-optimal performance |

Table 4: Effect of training set size.

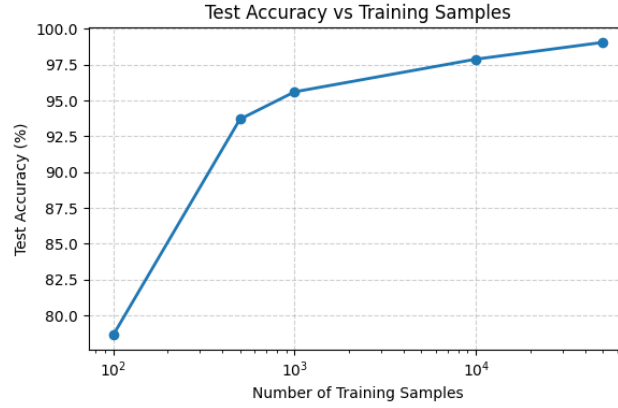


Figure 7: Test Accuracy vs. Number of Training Samples.

## References

- [1] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [2] C.-J. Hsieh, K.-W. Chang, C.-J. Lin, S. S. Keerthi, and S. Sundararajan. A dual coordinate descent method for large-scale linear SVM. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 2008.
- [3] J. C. Platt. Sequential minimal optimization: A fast algorithm for training support vector machines. *Microsoft Research Technical Report MSR-TR-98-14*, 1998.
- [4] B. Gassend, D. Clarke, M. van Dijk, and S. Devadas. Silicon physical random functions. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002.